Hands-on Activity 10.1	
Graphs	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed:** Sep 30, 2025
Section: CPE21S4	**Date Submitted:** Oct 2, 2025
Name(s): Punay, Heidee S.	**Instructor:** Engr. Jimlord Quejado

6. Output

```
#include <iostream>
// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph{
// insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head; // point new node to current head

        return newNode;
}
    int N; // number of nodes in the graph
public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
    // allocate new node
    head = new adjNode*[N]();
    this->N = N;
    // initialize head pointer for all vertices
    for (int i = 0; i < N; ++i)
    head[i] = nullptr;
    // construct directed graph by adding edges to it
    for (unsigned i = 0; i < n; i++) {
    int start_ver = edges[i].start_ver;
```

```
    int start_ver = edges[i].start_ver;
    int end_ver = edges[i].end_ver;
    int weight = edges[i].weight;
    // insert in the beginning
    adjNode* newNode = getAdjListNode(end_ver, weight, head[sta
    // point head pointer to new node
    head[start_ver] = newNode;
    }
}
// Destructor
~DiaGraph() {
    for (int i = 0; i < N; i++)
        delete[] head[i];
    delete[] head;
    }
};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";
    ptr = ptr->next;
    }
    std::cout << std::endl;
}

// graph implementation
int main()
{

    // graph edges array.
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
```

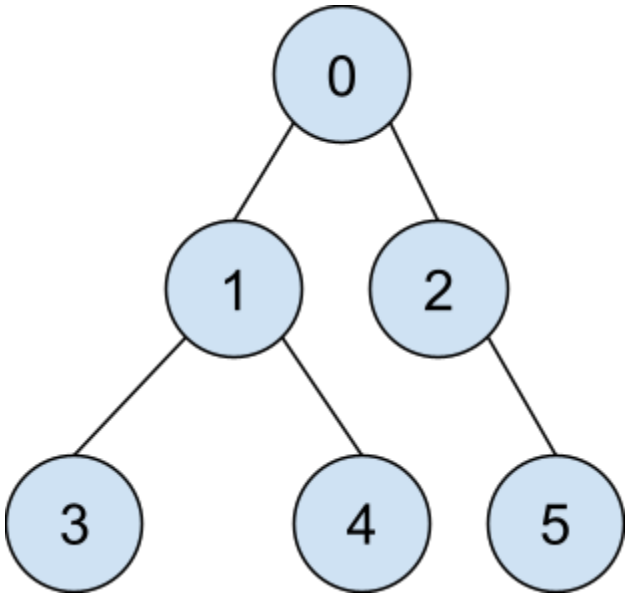
```
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
    {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;
    for (int i = 0; i < N; i++)
    {
    // display adjacent vertices of vertex i
    display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Output:

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)


--------------------------------
Process exited after 0.01751 seconds with return value 0
Press any key to continue . . .
```

Observation:

I had observed that Vertex 0 is the starting point; it then connects to 1 and 2. Vertex 1 connects to 0, 3, and 4, while Vertex 2, which was connected to Vertex 0, connects to 0 and 5. And vertices 3 and 4 connect to vertex 1. And lastly the vertex 5 is connected to 2.


0
1
2
3
4
5

B.1. Depth-First Search

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>
template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
        {
            os << i << ":\t";
            auto edges = G.outgoing_edges(i);
            for (auto &e : edges)
```

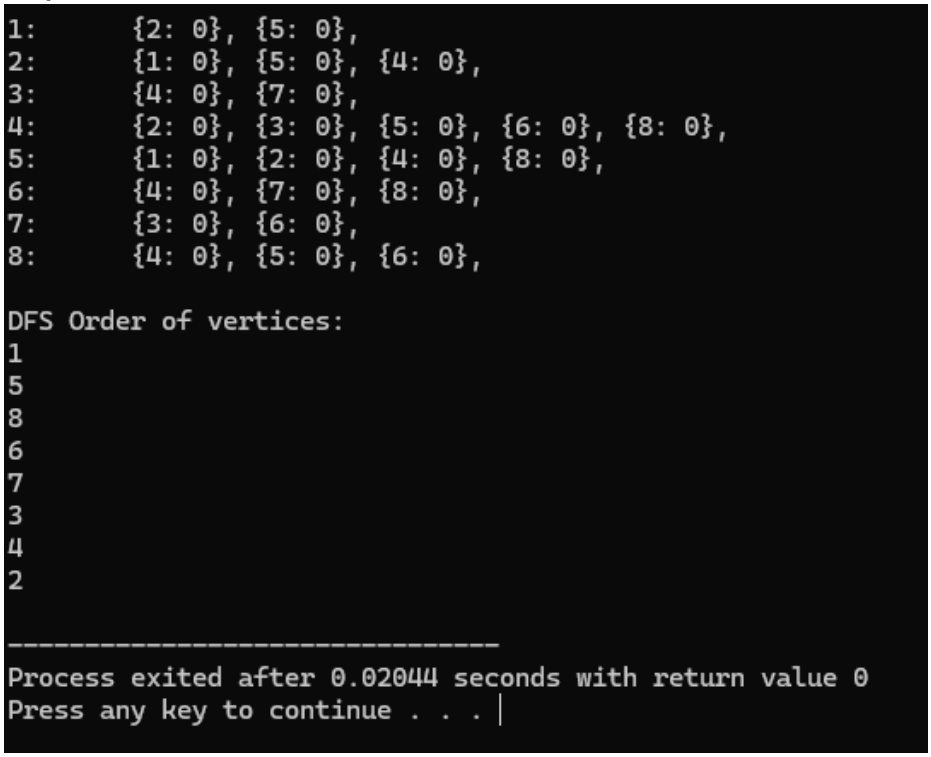
```
            os << "{" << e.dest << ": " << e.weight << "}, ";
            os << std::endl;
        }
    return os;
}

template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
    return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
    return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
    // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```cpp
// Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

// Overloads the << operator so a graph be written directly to a stream
// Can be used as std::cout << obj << std::endl;
template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
```

```
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
```

```
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
    }
    int main()
    {
        using T = unsigned;
        test_DFS<T>();
        return 0;
}
```

Output:



Observation:

I have observed that nodes are visited level by level. It explores deeply from the starting vertex before doing a backtracking. The depth-first search also uses a stack implementation, wherein it pops a vertex and pushes the neighbor into the stack.

B.2. Breadth-First Search

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
        {
            os << i << ":\t";
            auto edges = G.outgoing_edges(i);
```

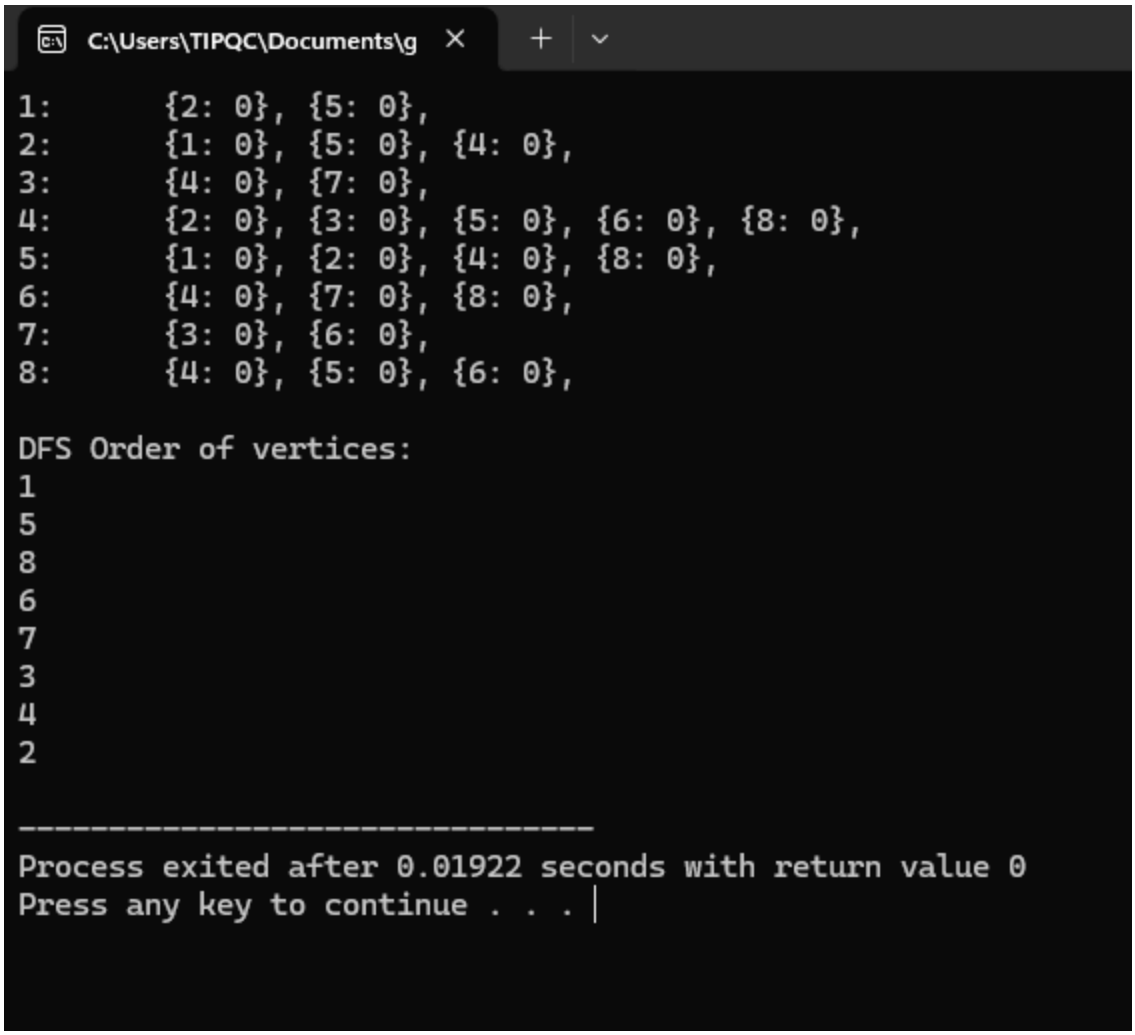
```
            for (auto &e : edges)
                os << "{" << e.dest << ": " << e.weight << "}, ";
            os << std::endl;
        }
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
```

```
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }


    template <typename T>
    friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

private:
        size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
```

```
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};


template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
```

```
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Output:



Observation:

In my observation, the breadth-first search visits all neighbors of a node before going much deeper. Like it starts from the vertex 0, then it first visits the 1 and 2 vertices (neighbor nodes) and continues to visit vertices 3,4, and 5 at the next layer.

7. Supplementary Activity

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.**

- The depth-first search would be most helpful in accomplishing the task. This algorithm is also how a person explores a map. It starts at one location and looks for other locations to visit, then backtracks to the other way to another location.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**

- The tree traversal is because it goes down to each branch completely before going to the next. The difference is only that in a tree, there are no cycles, and it is simpler compared to DFS in graphs.

3. **In the performed code, what data structure is used to implement the Breadth First Search?**

- It used a queue data structure where it follows the first-in, first-out rule. The starting node is enqueued, then each node's neighbors are also enqueued as they are discovered. Nodes are dequeued one by one, wherein the traversal visits each node level by level.

4. How many times can a node be visited in the BFS?

- In BFS, the nodes are visited only at once. Though a node may be enqueued many times if it is not checked, this algorithm ensures it is processed only once when nodes are visited.

8. Conclusion

To conclude, we learned how a graph is implemented and its algorithms, and how it is also closely similar to trees. It also has different algorithms to use in traversing the nodes or the vertices, the BFS and the DFS, wherein it has a different way of visiting each node. We also get to know the different parts of the graph and how it is plotted. The different algorithms have different data structure implementations, wherein the BFS uses a queue data structure and the DFS uses a stack data structure. Overall, the code is very hard to understand, but the concept of the graph and its algorithms are most likely not too difficult.

9. Assessment Rubric